

Unit 3

Unit 3: Angular, React, TypeScript — Practical/Oral Exam Notes

Context

Part of [Web Application Development](#) | Practical + Oral Exam Prep | SPPU Third Year IT

High-Yield Oral Topics

- **Angular Architecture** — Modules, Components, Services, DI
- **Data Binding** — 4 types, especially two-way `[(ngModel)]`
- **Directives** — `*ngIf` vs `*ngFor` (structural vs attribute)
- **React Hooks** — `useState`, `useEffect`, `useContext`
- **Angular vs React** — Always compared in viva
- **MVC Architecture** — How it maps to frameworks

MINI PROJECT ANCHOR: Student Management System (SMS)

Angular App: Register Student → Login → View Profile/Dashboard

SECTION 1 — MVC ARCHITECTURE

Exam Definition

MVC (Model-View-Controller) is a **software design pattern** that separates an application into three interconnected components to organize code and separate concerns.

Layer	Role	SMS Example
Model	Data + Business Logic	Student schema, DB queries
View	UI shown to user	HTML templates, Angular components
Controller	Bridge — processes requests	Express route handlers

MVC Flow (Viva Answer)

"User submits a registration form (**View**). The form data goes to the **Controller** which validates it and calls the **Model** to save to MongoDB. The Model returns success. The Controller sends a response. The View updates to show the new student."

MVC in Angular

- **Model** = TypeScript interfaces/classes (`student.model.ts`)
- **View** = Component template (`.component.html`)
- **Controller** = Component class (`.component.ts`)

SECTION 2 — TYPESCRIPT

📌 Exam Definition

TypeScript is a **statically typed superset of JavaScript** developed by Microsoft. It adds type safety and OOP features, then **compiles to plain JavaScript**.

Key Features

Key concepts:

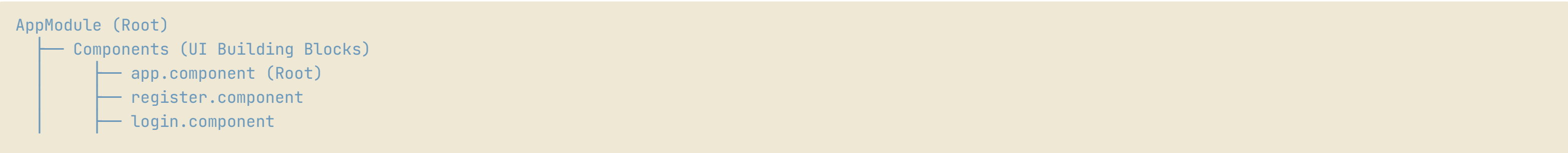
- **Static Typing**: Explicitly declaring types for variables and functions (e.g., `let name: string`).
- **Interfaces**: Defining the structure of an object.
- **Classes**: Support for full object-oriented programming.
- **Modules**: Using `export` and `import` to organize and share code between files.

JS vs TypeScript

Feature	JavaScript	TypeScript
Typing	Dynamic (runtime errors)	Static (compile-time errors)
OOP	Partial	Full (interfaces, generics)
IDE Support	Basic	Excellent autocomplete
Compilation	Runs directly	Compiles to JS first
Error Detection	Runtime	Compile time

SECTION 3 — ANGULAR

Angular Architecture Diagram



```
└── profile.component
└── Services (Shared Logic + HTTP)
    └── student.service.ts
└── Routing (app-routing.module.ts)
└── Models (student.model.ts)
```

Angular CLI Commands — Must Know

```
# Install Angular CLI globally
npm install -g @angular/cli

# Create new project
ng new student-management-system
cd student-management-system

# Generate components
ng generate component register
ng generate component login
ng generate component profile

# Generate service
ng generate service student

# Run development server
ng serve           # Runs on http://localhost:4200

# Build for production
ng build --prod
```

SMS Angular — Complete Register Component

An Angular component class handles the logic. It uses **Dependency Injection** in the constructor to acquire services like `StudentService` and `Router`. It defines methods like `onSubmit()` to handle user actions.

The template defines the view. It uses **Directives** like `*ngIf` for conditional messages and `[(ngModel)]` for two-way data binding on form inputs, linking them directly to the component class properties.

Student Service — HTTP Calls

Services in Angular are used to share logic or data across components. They often use `HttpClient` to make REST API calls and return `Observable` streams that components can subscribe to.

Angular Routing Setup

Routing allows navigation between different views. Routes are configured as an array of objects mapping a URL `path` to a specific `component`.

```
<!-- app.component.html - Router Outlet -->
<nav class="navbar navbar-dark bg-dark">
  <a routerLink="/register" routerLinkActive="active">Register</a>
  <a routerLink="/login" routerLinkActive="active">Login</a>
  <a routerLink="/profile" routerLinkActive="active">Profile</a>
</nav>

<!-- Component renders here based on route -->
<router-outlet></router-outlet>
```

Data Binding — 4 Types (MUST KNOW)

Type	Syntax	Direction	Example
Interpolation	<code>{{ value }}</code>	Component → View	<code>{{ student.name }}</code>
Property Binding	<code>[property]="value"</code>	Component → View	<code>[disabled]="isLoading"</code>
Event Binding	<code>(event)="method()"</code>	View → Component	<code>(click)="onSubmit()"</code>
Two-Way Binding	<code>[(ngModel)]="var"</code>	Both ways	<code>[(ngModel)]="student.name"</code>

Directives Quick Reference

- `*ngIf`: Conditionally includes an element based on a truthy/falsy expression.
- `*ngFor`: Renders a list of items by iterating over an array.
- `[ngClass]`: Dynamically adds or removes CSS classes.
- `[ngStyle]`: Dynamically sets inline styles.

Lifecycle Hooks

Hook	When it fires	Use Case
<code>ngOnInit</code>	After component created	API calls, initialization
<code>ngOnChanges</code>	When <code>@Input()</code> value changes	React to parent data
<code>ngOnDestroy</code>	Before component removed	Cleanup subscriptions

Components use hooks like `ngOnInit` to fetch data when the component **is** created, and `ngOnDestroy` to clean up resources like subscriptions to prevent memory leaks.

SECTION 4 — REACT (OVERVIEW)

React Hooks — SMS Example

- **`useState`**: Manages state in a functional component (e.g., `[students, setStudents]`).
- **`useEffect`**: Handles side effects like data fetching. The dependency array `[]` ensures it runs once on mount.
- **`JSX`**: Allows writing HTML-like structure directly in JavaScript.

Angular vs React — MOST ASKED COMPARISON

Feature	Angular	React
Type	Full Framework	UI Library
Language	TypeScript	JavaScript/JSX
DOM	Regular DOM	Virtual DOM
Data Binding	2-way by default	1-way by default
State Management	Services + RxJS	useState + Redux
Learning Curve	Steep	Moderate
Opinionated	Yes (strict structure)	No (flexible)
Backed by	Google	Meta (Facebook)
Best For	Enterprise apps	Dynamic SPAs

VIVA Q&A — Unit 3

Q1: What is Dependency Injection in Angular?

DI is a design pattern where Angular automatically provides instances of dependencies (like services) to components that need them. Instead of creating `new StudentService()` manually, you declare it in the constructor and Angular injects it. Benefits: Loose coupling, easier testing, single instance (singleton) across app.

Q2: What is `*ngFor` vs `*ngIf` ?

Both are **structural directives** that modify DOM structure. `*ngIf` conditionally shows/removes an element. `*ngFor` repeats an element for each item in an array. They both use the `*` prefix which means they can add/remove DOM elements.

Q3: What is a Service in Angular?

A Service is a class with `@Injectable()` decorator that contains **shared business logic** (like HTTP calls). Services are **singleton** — one instance is shared across all components. They're injected via DI. Example: `StudentService` with `register()`, `login()`, `getAll()` methods used by multiple components.

Q4: What is the difference between `Observable` and `Promise`?

`Promise` handles a **single** async event. `Observable` handles **streams** of events over time and can be cancelled. Angular's `HttpClient` returns `Observables`. You subscribe with `.subscribe()`. `Observables` support operators like `map`, `filter` from RxJS.

Q5: Explain `useState` and `useEffect` in React.

`useState(initialValue)` gives a component memory — it returns `[value, setter]`. Changing value via setter re-renders the component. `useEffect(fn, deps)` runs side effects after render — for API calls, DOM updates, subscriptions. The `deps` array controls when it runs: `[]` = once on mount, `[var]` = when var changes.

⚠ Oral Trap Questions

Trap: "What is `routerLink` vs `href` in Angular?"

`href` causes a full page reload — it's for regular links. `routerLink` is Angular's directive that navigates WITHOUT reloading the page. It uses the Angular Router to load the component dynamically. Always use `routerLink` inside Angular apps.

Trap: "Can you use `*ngIf` and `*ngFor` on the same element?"

No! Angular doesn't allow two structural directives on the same element. Solution: wrap in an `<ng-container>` which renders no extra DOM element.

📋 PRACTICAL FILE — Assignment 2 (Angular Part)

Viva explanation:

"I created an Angular application with 3 components: Register, Login, and Profile. The Register component uses two-way data binding `[(ngModel)]` to bind form fields to a TypeScript object. On submit, it calls `StudentService.registerStudent()` which uses `HttpClient.post()` to call our Node.js REST API. The Router navigates to Login after success. The Profile component loads all students using `ngOnInit` and displays them with `*ngFor`."

🔥 High Probability Questions

1. Explain Angular Architecture with diagram
2. What are the 4 types of data binding in Angular?
3. Difference between structural and attribute directives
4. What is Dependency Injection? Why use it?
5. Explain `ngOnInit` lifecycle hook
6. What is `[(ngModel)]`? How does two-way binding work?
7. Angular vs React comparison

8. What is `useState` in React?

9. What is a Service in Angular?

10. What is routing in Angular? How to configure it?